Practical AI for Teaching

Session 1 – an introduction to generative AI

Claes Bäckman
Leibniz Institute for Financial Research SAFE

Three messages: AI is here, the workflow matters more than the model, and verification is key.

1. **The tools changed.** Chat gave way to agents that read files, run code and iterate. The tool you tried in 2023 is not the tool of today.
2. **The workflow is the value.** The agent's leverage comes from context (your files), standing instructions, and reusable skills — not from raw model IQ.
3. **Judging the output is the new job, for you and for your students.** Silent errors and confident statements with no backing are what to look out for, in your own work and in what students hand in.

Workshop synthesis, drawing on Mollick (2026), Velikov (2026) and Korinek (2025).

Six topics this morning, hands-on this afternoon

1

Fundamentals and tools

How the tools evolved, and Copilot compared with the rest.

2

Supervision and assessment

AI support for theses, what detectors can do, an AI-written thesis.

3

Teaching in general

Protect what must be learned, then teach more.

4

Skills

Thesis feedback as a skill. Writing and updating your own.

5

Replication and coding

A student's replication package, and how to check the code.

6

Risks

Hallucination, copyright and plagiarism.

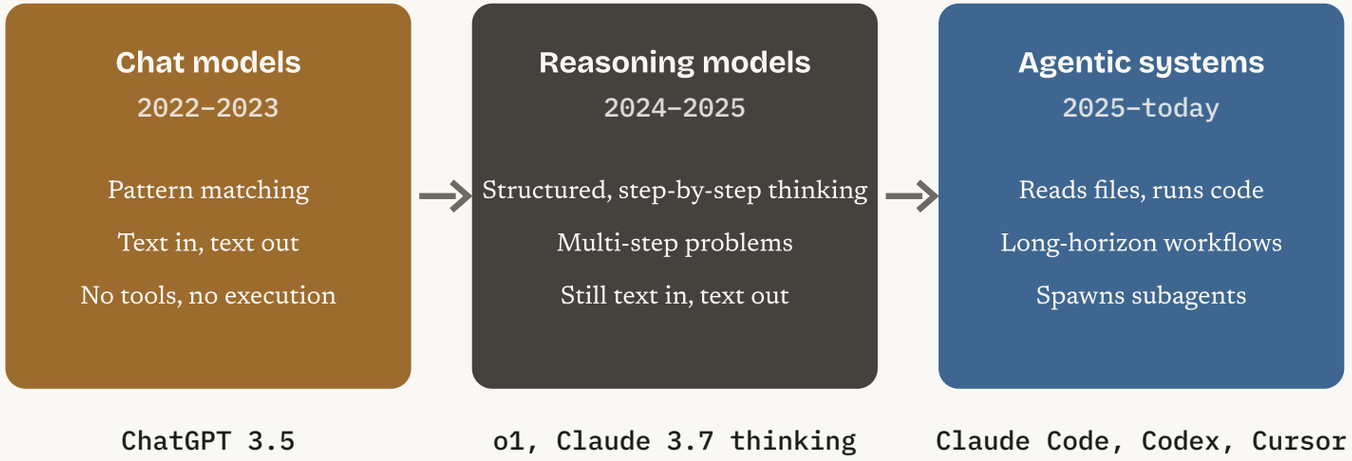
Afternoon. Hands-on with your own materials: feedback on a thesis, a skill built from your own rubric, a replication package, a course update.

Live example: an agent starts a thesis now, and we check on it in section 2

> *I want to write a paper on AI exposure and house prices in Denmark. My hypothesis is that areas with greater AI exposure have faster house price growth. You can download data using the DST API document here: https://www.dst.dk/en/Statistik/hjaelp-til-statistikbanken/api I first want you to make a plan for how you would do this, including how to get the data and how to analyze it, and then we will iterate on the plan. Afterwards I want you to write up the results in a paper.*

1 · Fundamentals and tools

Three generations in four years: chat, reasoning, agents

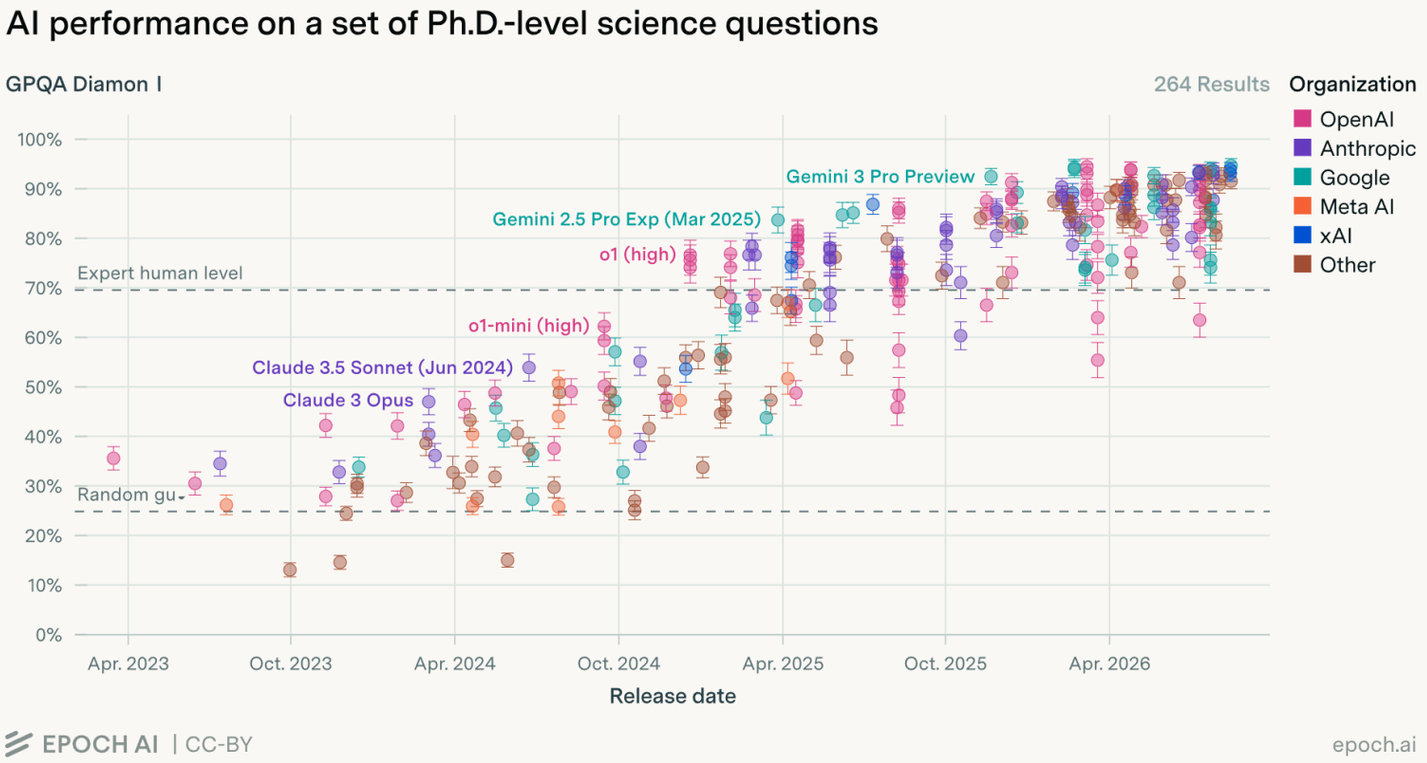


After Mollick (2026), Korinek (2025) and Velikov (2026).

A brief example on developments since 2023

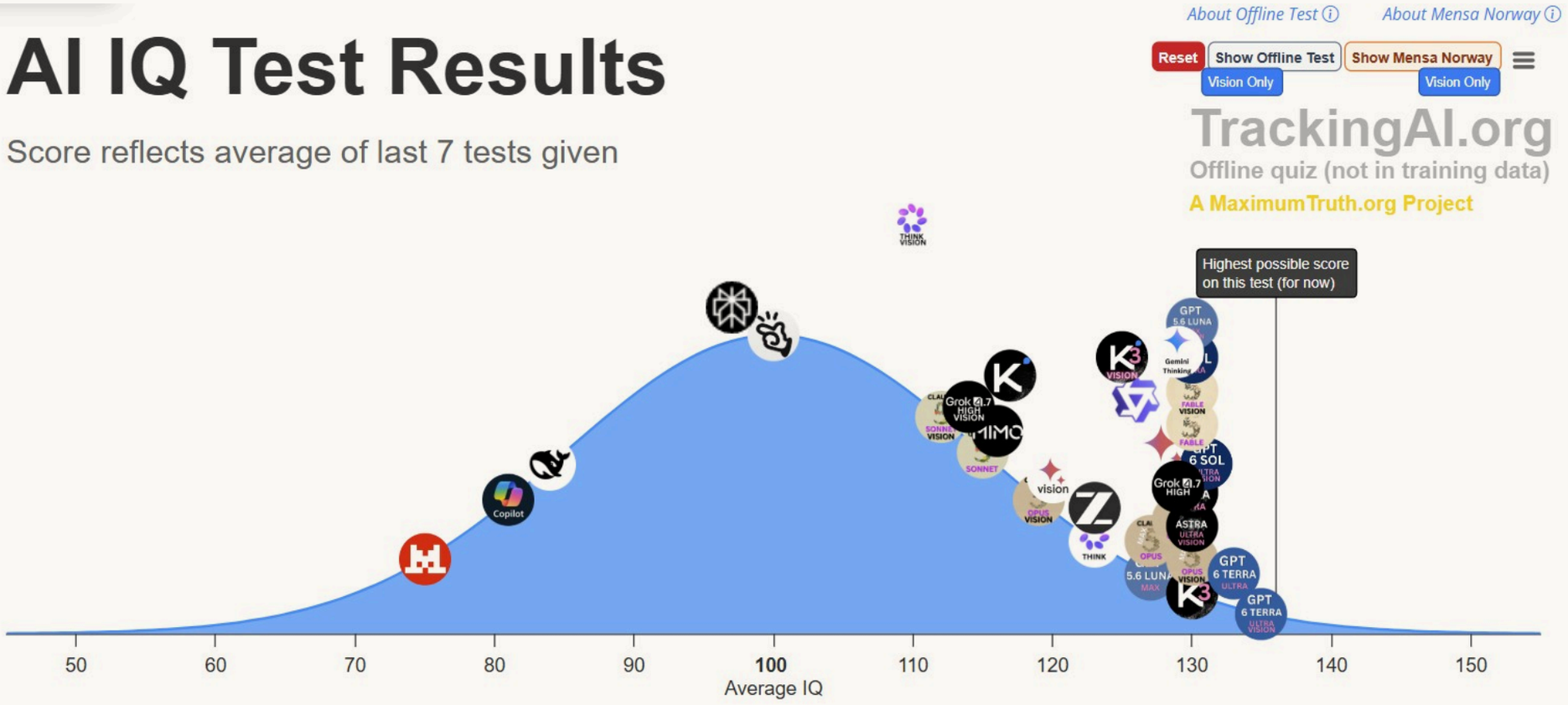
> Can you give me a summary of the paper by Olga Balakina, Charlotte Christiansen, and Malene Kallestrup-Lamb? Don’t use web search

On PhD-level science questions, models are now above expert level

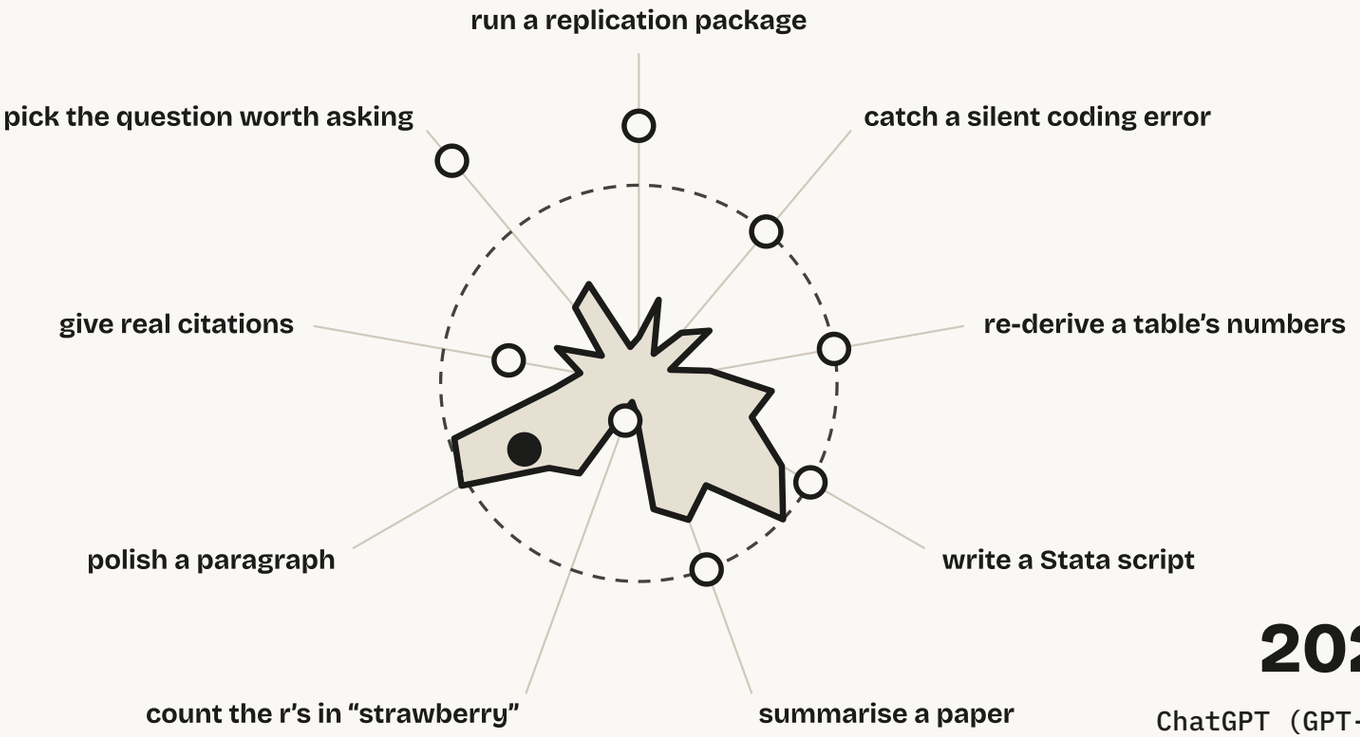


GPQA Diamond accuracy by model release date, 264 results; each dot is one model. Dashed lines: random guessing (25 percent) and expert human level (about 70 percent). Epoch AI, CC-BY.

Models differ



But the frontier is jagged



2022

ChatGPT (GPT-3.5)
1 of 9 tasks inside

inside the frontier: the AI does it — the frontier in earlier years - - tasks equally hard for a person

Next year | 2022 | 2023 | 2024 | 2025 | 2026

ChatGPT polishes a paragraph. It cannot take in a whole paper, invents its citations and miscounts letters.

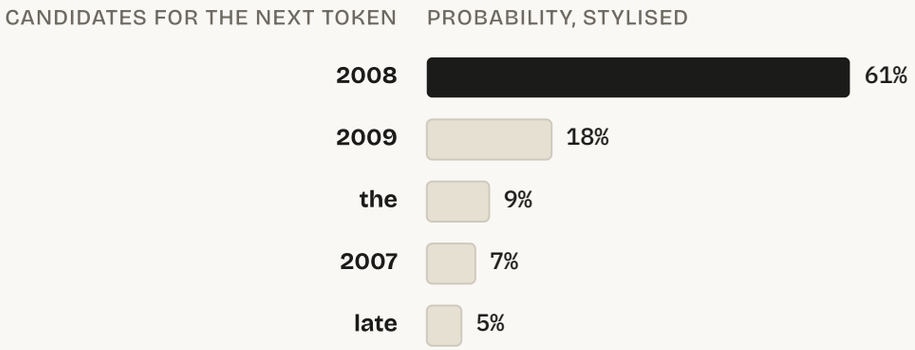
Stylised, after Mollick (2023) and Dell'Acqua et al. (2026). Distance from the centre is how hard a task is for a person. Which year each task moves inside is the deck's reading of model releases, not a measurement.

Who uses Copilot Chat or ChatGPT? Who has used an agent, such as Claude Code or GitHub Copilot in VS Code?

An LLM predicts the next token, over and over, very well

A token is a word or a piece of a word, about four characters of English. That is the whole training objective: given the text so far, predict the next token. Press the button.

```
House prices in Denmark fell sharply in
```



Sample the next token | Reset | Press **sample**: the most likely token is appended, then the model predicts again.

Candidate lists and probabilities are invented for the slide; a real model has tens of thousands of candidates. The mechanism is the real one: predict a distribution over the next token, pick one, repeat. The stage always picks the top candidate; a real model samples.

So it is a prediction machine, not a lookup machine

Not a database

It does not look facts up. It generates text that is plausible given what it has seen.

Not a calculator

It predicts what an answer looks like. The arithmetic is right when the pattern is common, and the tools in section 6 exist because of this.

Not connected to your files

Or to the live web, unless the tool around it adds that. Out of the box it knows nothing about your course.

An invented citation is a sequence of plausible tokens. “Smith and Jones (2019)” is very plausible.

A very well-read, very fast colleague who has read almost everything but remembers none of it precisely, has no access to your files, will never admit to not knowing, **and is** unapologetic about being wrong.

Different tools

Copilot Chat	`KTH approved`	Microsoft's chat tool for staff and students, recommended by KTH for its data protection
ChatGPT	`KTH approved`	For staff, through KTH IT; not for students
GitHub Copilot	`separate`	An agent in VS Code; it reads the same skill files as Claude Code
Claude Pro	`$20 / month`	Claude Code, what session 2 runs on. Not on KTH's list: your own material only, never student work **SESSION 2**

KTH's notice on approved AI services (22 December 2025) and its FAQ on generative AI. Claude's tiers as of August 2026; Max, at $100–200 a month, is for heavy use.

Copilot Chat sees only what you paste, Claude Code the whole project folder

	COPILOT CHAT	CODEX
Where it runs	On Microsoft's servers: browser, Teams, Outlook	In a project folder on your computer, from VS Code
Sees your files	Only what you paste or upload	Any file in the folder, read when the task needs it
Runs code	At most Python in Microsoft's sandbox, not on your machine	Python, MATLAB or LaTeX on your computer, after asking you
Standing instructions and skills	Custom instructions and memory. Skills from October to November 2026, if KTH IT allows	`AGENTS.md` in the folder, read at every start. Skills in `.codex/skills`, run with `$name`
KTH status	Licensed by KTH, with enterprise data protection	Available through ChatGPT Edu

Chat runs in a sandbox; an agent runs on your computer

Chat (Copilot Chat, ChatGPT, Claude.ai)

- Runs on the provider's server, walled off from your machine
- Sees only what you paste or upload
- Gives text back; you move it where it belongs
- Answers get generic when the paste is thin

When to use: brainstorming, polishing one paragraph, quick lookups.

An agent (Claude Code, Codex, GitHub Copilot)

- Runs in a folder on your computer
- Can read any file in the project, and edit it
- Runs code, shell commands, search, web, MCP tools
- Output is grounded in your files and lands in them

When to use: any task that touches more than one file or needs to execute code.

But the two are converging

What chat borrowed from agents

Claude and ChatGPT both have **projects**: upload files, set standing instructions, keep a memory per project. That is the context half of what an agent has, in a chat window.

Both apps now contain an agent: the *Code* and *Cowork* tabs in the Claude app, Codex inside ChatGPT.

What chat still lacks

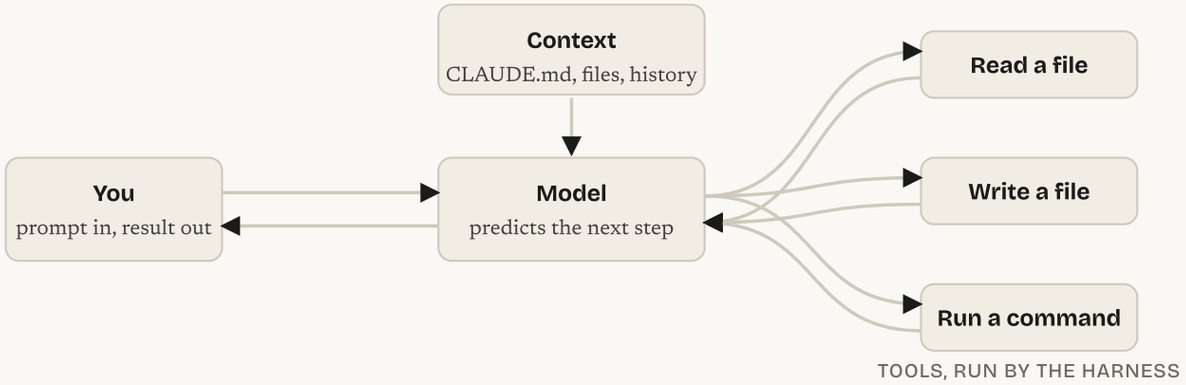
The loop. Projects hold context; they do not read your folder, write files, or run code and fix what broke.

The agent is a loop: prompt, model, tool, result, repeat

Your prompt goes to the model together with the context: CLAUDE.md, a file of standing instructions the agent reads every session, plus the folder listing and the conversation so far.

The model decides which tool to call. The harness runs it and puts the result back into the conversation. Repeat until the task is done, or the agent asks you a question.

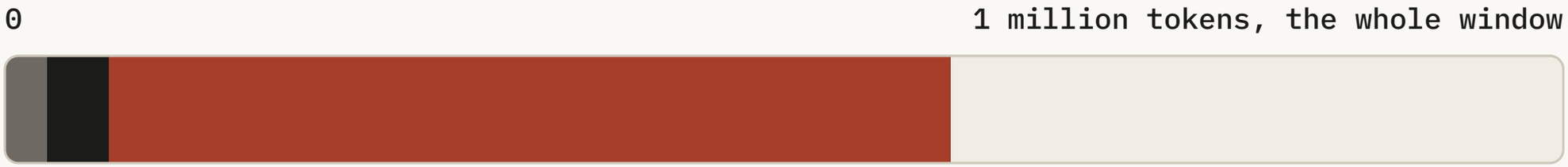
Stylised trace of the course-page session. After Korinek (2025); figure adapted from Velikov (2026). All six tools are in the appendix.



Step Reset Press **step** to run the request through the loop.

Context is a budget

Everything the model conditions on has to fit in one window: the standing instructions, the tool definitions, the conversation, and every file it reads. Drag the slider to the right.



system prompt, CLAUDE.md, tool definitions · conversation so far · files the agent has read

PDF pages read this session 1200

1,200 pages, about 6 long PDFs ≈ 540,000 tokens, 61% of the window used in total.

Stylised arithmetic with real orders of magnitude: about 1 million tokens in the window, about 450 tokens a PDF page, so a 200-page PDF is 80–100k. The fixed segments are rules of thumb for a Claude Code session. Quality degrades well before the bar is full.

Context improves AI tools tremendously, and the goal is to provide the right context.

Chat: you paste it. Projects: you upload it once. An agent: it goes and reads it.

Five prompting rules: specific, iterate, correct early, verify, restart

1

Specific

“Feedback on the hypotheses in section 2, against the rubric”, not “feedback on this thesis”.

2

Iterate

Ten to twenty turns, not one. Each turn adds context.

3

Correct early

If the first answer is off, fix it now, or restart.

4

Verify

Run the code. Re-read the passage. Check the citation.

5

Restart

Thirty turns or a shifted task: a fresh session.

2 · Supervision and assessment of theses

Students already use AI for assessed work, mostly without guidance

94%

of UK undergraduates use generative AI to support assessed work

48%

say their teaching staff help them build the AI skills they need

35%

of students in Sweden used ChatGPT regularly in spring 2023, five months after its release

Higher Education Policy Institute and Kortext (2026), 1,054 UK undergraduates, December 2025. Malmström et al. (2023), 5,894 students in Sweden, April–May 2023, recruited through social media and student organisations.

The research plan for the AI and house prices project: question, data, design and what could go wrong

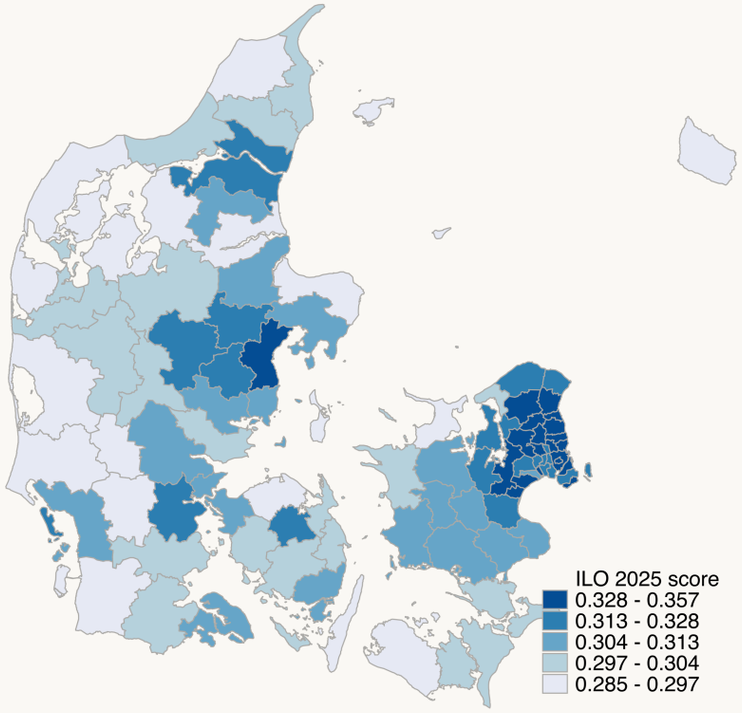
1. **Question.** Did house prices grow faster after ChatGPT in Danish municipalities whose residents hold AI-exposed jobs?
2. **Data.** 14 tables from Statistics Denmark and house prices from Finans Danmark, for 98 municipalities.
3. **Design.** Exposure from ILO occupation scores, fixed before 2022, and an event study that compares municipalities within a province.
4. **Threats.** The urban trend, the 2022 rate shock, the 2024 property-tax reform, remote work.

From the plan, in the agent's words

"RAS301 counts jobs located in a municipality and RAS302 counts employed residents. The table titles do not say this, so I checked the counts. In 2024 RAS301 records 464,753 employed in Copenhagen and RAS302 records 395,027."

The research plan in the thesis example, sections 1–5. The live session on screen is at an earlier stage of the same prompt.

Five minutes of my time and 30–45 minutes of the agent's: the data, a map and a first regression



Exposure of employed residents, 2017-2019 employment shares. Quintiles. Bornholm is moved north-west.

5 min

of my time: the prompt and a few replies

30–45 min

of the agent working on its own

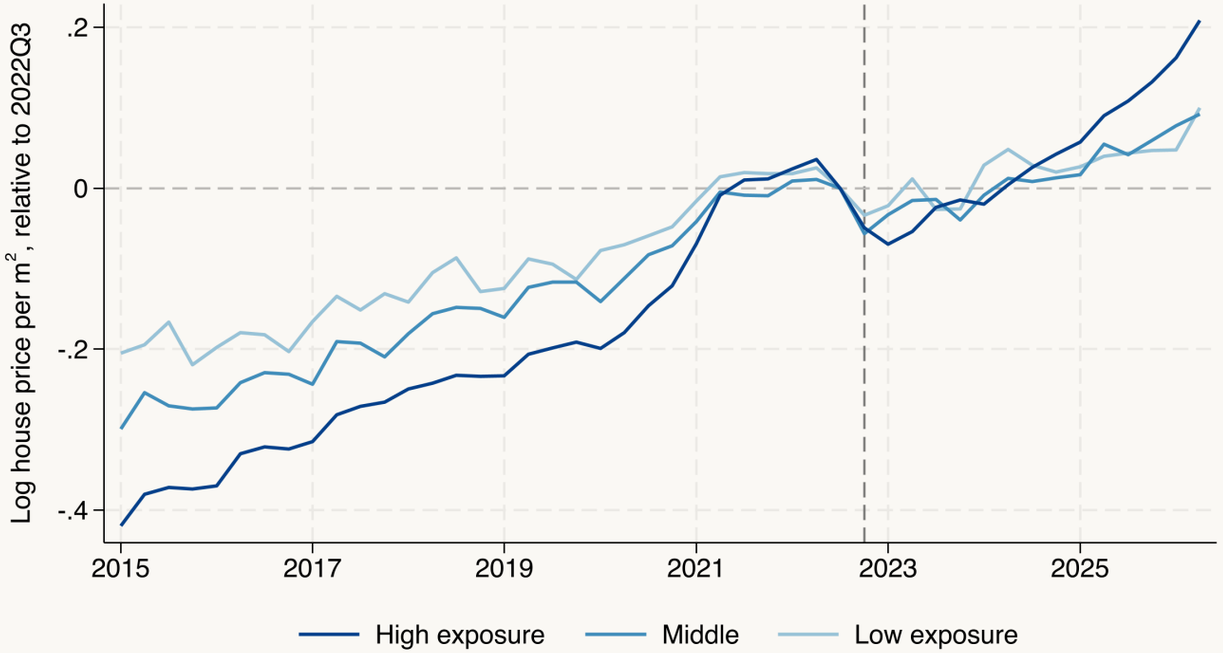
≈ 0

extra cost beyond the subscription

In the folder. Nine scripts in Python and Stata and one file that runs them in order, data from Statistics Denmark, Finans Danmark and the ILO, four figures, three tables, and the plan with the results written up.

AI exposure of employed residents in Denmark's 98 municipalities, from the thesis example. Time and cost from the run.

The agent reports the bad news itself: the gap between exposed and unexposed areas predates ChatGPT



Unweighted means across municipalities in each tercile of resident AI exposure.
Dashed line: release of ChatGPT (2022Q4).

“The raw association predates ChatGPT.”

“Within provinces there is no association yet.”

High-exposure municipalities gained about 20 log points on low-exposure ones between 2015 and 2021. Within provinces the estimate after ChatGPT is 0.8 log points, with a standard error of 1.0.

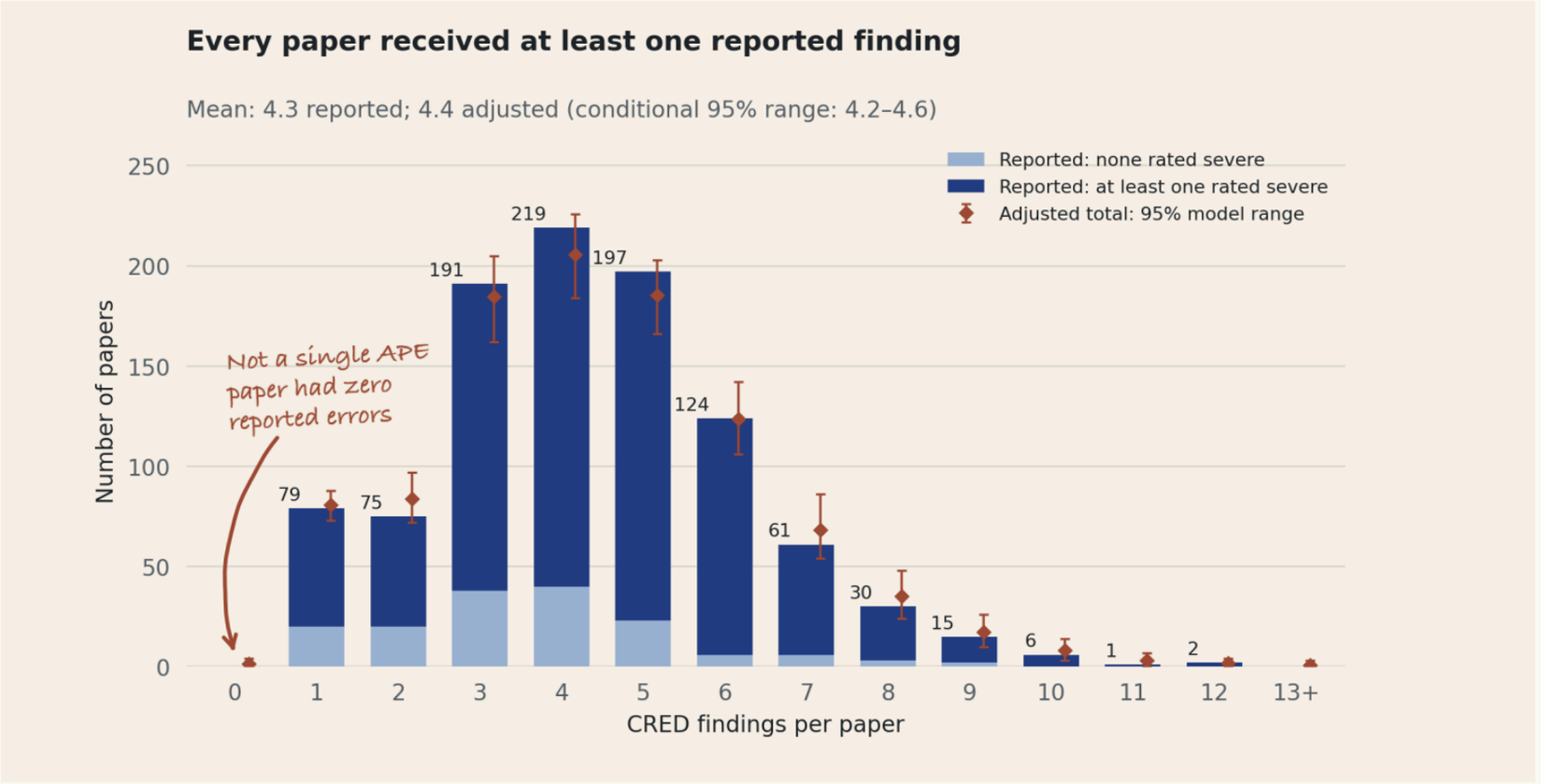
The agent’s summary of its own results, research plan section 9. House prices per m² by tercile of AI exposure, relative to 2022Q3.

Efforts underway to automate research

1. Project APE has automated 1,000 policy-evaluation papers.
2. Alejandro López-Lira built a system that finds a problem, generates a theory, verifies it adversarially and writes a publication-ready paper.
3. Novy-Marx and Velikov (2026) describe how academic research could be automated with LLMs, and what that does to the literature.

For a supervisor: a document that looks like a thesis is no longer evidence of the work behind it.

Are we there yet?



“We conclude that, in the first half of 2026, our autonomous policy evaluations remained unreliable despite our best efforts with frontier models and research workflows designed for reliability.”

APE Briefs, October 2026

How to evaluate this as a project?

1. A choice. “Why measure exposure where people live and not where they work?”
2. A number. “Exposure correlates 0.94 with the 2019 house price. What does that do to your estimate?”
3. A check. “Which step did you check by hand, and what did you find?”
4. An alternative. “The gap reopens in 2024, when interest rates fell. How do you tell AI from rates?”

A student who did the work, with or without an agent, can answer these. One who only pressed enter cannot.

Questions written from the thesis example’s own plan and results.

You can ask AI to prepare the questions

> *Read the thesis in this folder. Write five questions for the opponent to ask at the defence, each answerable in under a minute without the text: one on the research question, one on the data, one on a specific table or figure, one on a choice the student made, and one on what the result does not show. For each, say what a good answer contains.*

Why it works. The questions come from the thesis itself. A student who understood their thesis can answer them, and one who did not cannot.

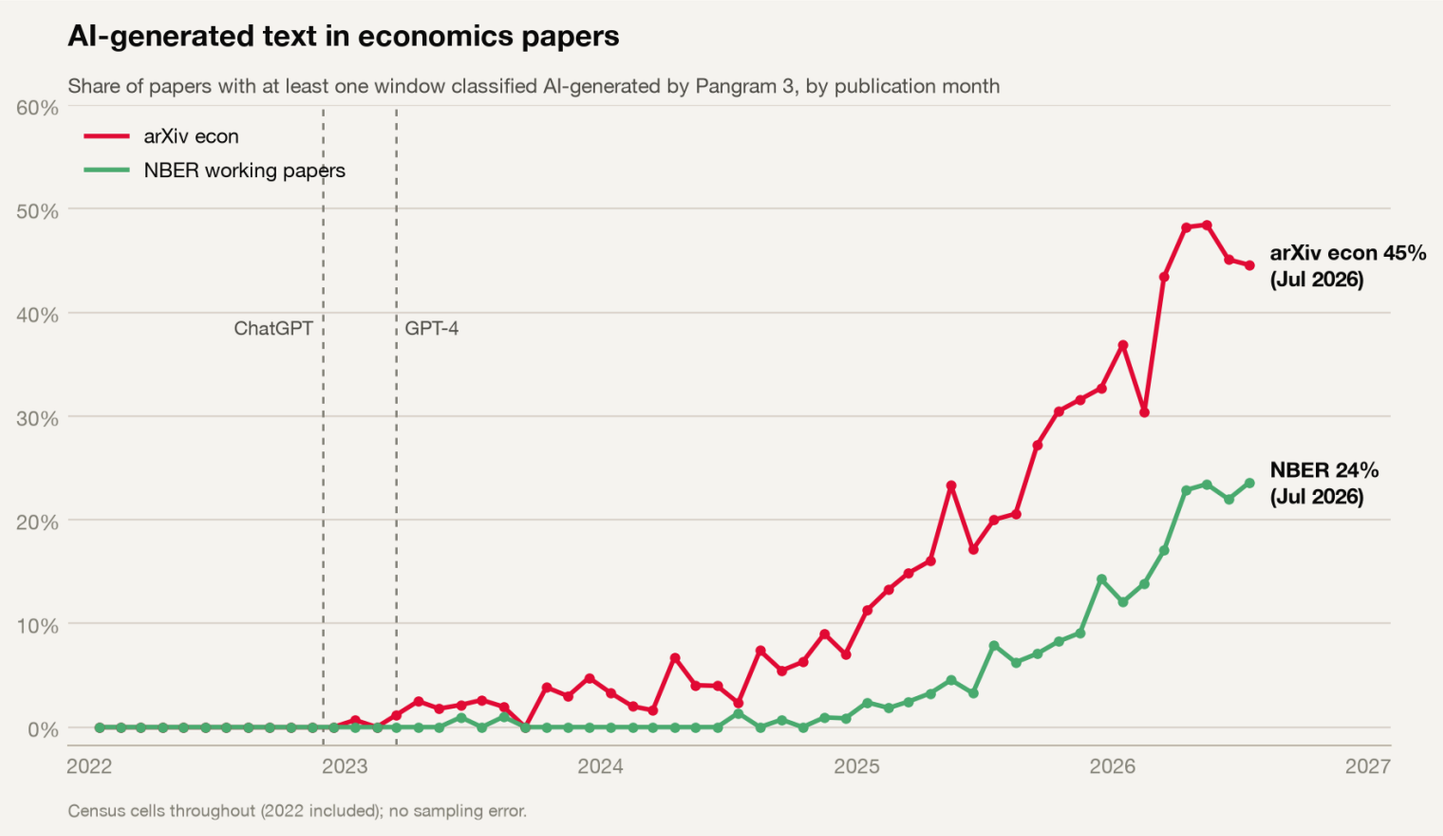
What to check. Every question must point at something that is really in the thesis. Delete the generic ones.

On a real thesis, the student could also run it. The author could prepare with it, for good or for bad.

Students could also prepare using this technique

> *I want to make sure I know the paper in this folder. Make a html site with quiz questions for both the paper and the presentation based on what an finance department at a top university would ask me. You can use subagents. Make both easy and hard questinos, both on practical details and conceptual things, and have one agent try to go beyond the paper and ask what it means for finance theory or in the context of finance theory. Ask questons if you want.*

AI-written passages now appear in a quarter of new NBER papers



Share of papers with at least one passage the Pangram detector classifies as AI-generated, by month of publication. Figure from Paul Goldsmith-Pinkham, A Causal Affair.

The best detectors are now very good on English text, but detectors differ a lot

26%

of AI text caught by OpenAI's own detector in 2023, while it flagged 9% of human text. Withdrawn after six months

≈ 0

false alarms and misses for the best detector, Pangram, in a 2025 audit of medium and long passages

~50%

of AI text missed by GPTZero once a "humanizer" tool had rewritten it. Pangram still caught it

Students should not assume that AI text goes unnoticed. Which detector is used matters as much as whether one is used.

OpenAI (2023). Jabarian and Imas (2025), 1,992 English passages in six genres from four frontier models. A working paper, not yet peer reviewed.

Let’s try it out on the AI generated thesis.

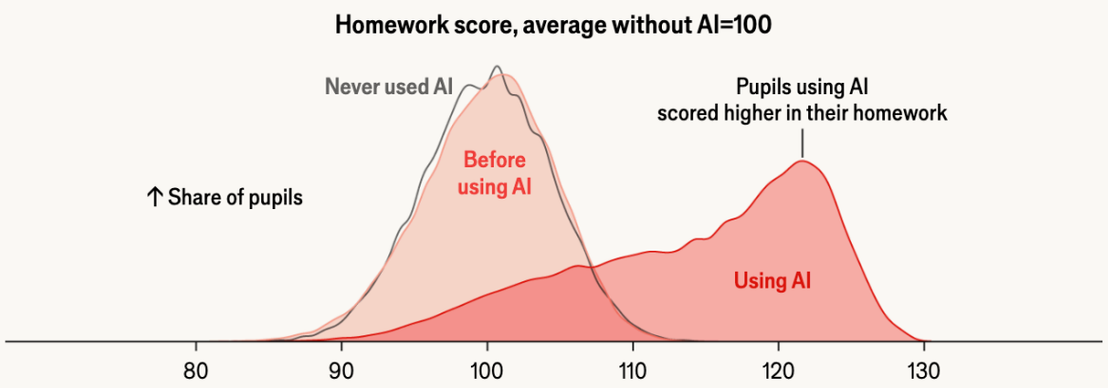
3 · Teaching in general

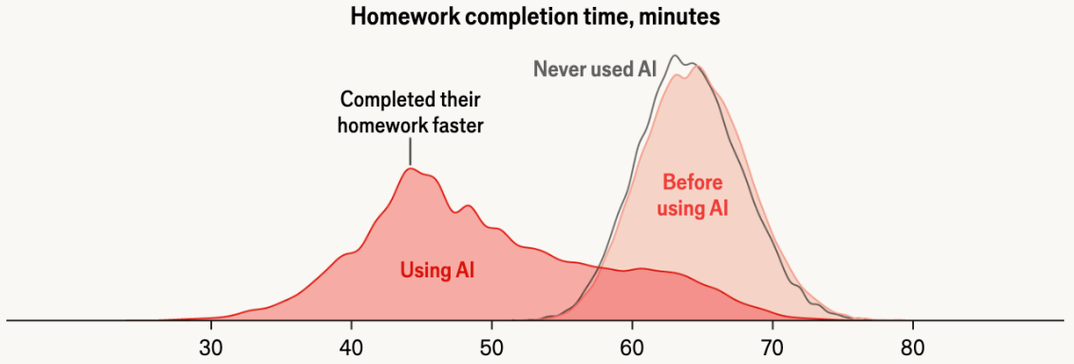
The challenge with AI: take-home grades stopped measuring learning



One course (Serrano, ECON 1170)

With AI, homework got better and faster





+18%

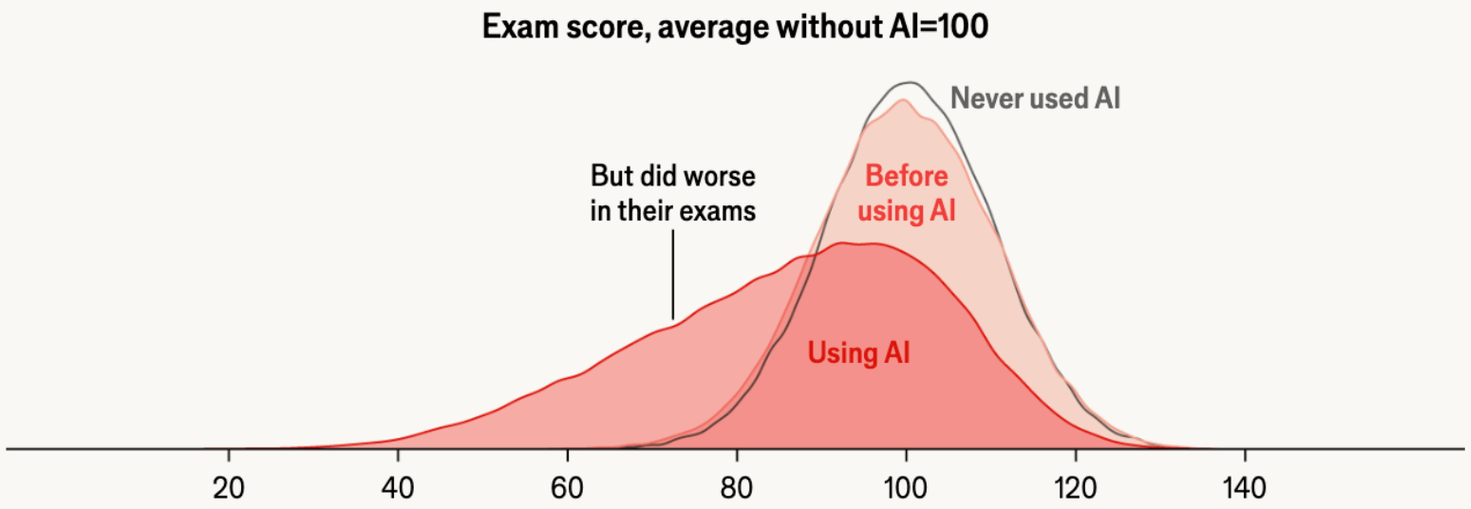
homework score for pupils using AI

−30%

homework completion time

Distributions for pupils in China: never used AI, before using AI, and using AI. Charts reproduced as published; study as cited in Bryan (2026).

But exam scores fell



−20%

exam performance for the same pupils

Homework up, time down, exams down. Pupils who used AI on homework did worse on the exam.

Same study and source as the previous slide.

AI breaks the grade–knowledge link, so the rules have to change

1. **The grade–knowledge link is broken.** A good take-home grade no longer shows the student can do the thing.
2. **Students need to use AI well later.** Banning it everywhere leaves them untrained.
3. **AI should let us teach more.** Tutoring, feedback and individual practice used to be too expensive.

Bryan (2026), kevinbryanecon.com/teachwithai.

Protect learning: decide, test, do not assign what AI can do

1. **Decide what should be learned without AI.** Be explicit about which assignments test AI-independent competence.
2. **Give students an idea of how AI impacts learning.** Everyone is struggling through what AI means for teaching, including students.
3. **Give students an incentive to learn.** A closed book exam gives incentives to learn.
4. **Do not give AI-cheatable assignments.** Almost every take-home worth real credit can be done by a frontier model today.

1, 3, and 4 from Bryan (2026).

AI rewards expertise that the students do not yet have

What the expert brings

Better questions. An expert asks the question that is worth asking.

Steering. An expert knows which follow-up moves the conversation where it should go.

Validation. An expert can tell when the output is wrong.

Teaching more with AI

For the students

Help them learn more efficiently. Class-specific tutoring, spaced repetition, mastery learning at scale.

Personalise assignments. Problems at the level each student is struggling with.

For the teacher

Improve your own teaching. Find out where you were confusing before the exam tells you.

Raise standards. If research and drafting are cheaper, the bar moves up.

Rules 4–7 of Bryan (2026).

Three things you can try

Course tutor

A shared assistant (a Claude project, custom GPT or NotebookLM) loaded with *your* lecture notes and problem sets.

Standing instructions: use the course's notation, never give the answer, ask the student to explain first, return to topics they missed.

Misconception digest

Each week, export the questions students asked, anonymised.

Prompt: "Group these by underlying misconception, rank by frequency, and point to the lecture slide most likely to have caused each."

Levelled problem sets

One problem, three difficulty tiers. Assign the tier by last week's quiz result.

Prompt: "Write this problem at three levels: scaffolded, standard, extension. Same concept, full solutions, and a rubric."

Student questions are personal data. Anonymise before exporting and check the institutional policy once.

AI is going to change things

Same teaching, done faster

Course materials. Slides with current examples, problem sets with solutions, exams with rubrics, code examples in Stata, R or Python.

Feedback. Students run your rubric on their own draft before every seminar, so they arrive with the obvious problems fixed.

New things, not only old things faster

New ways to teach. A course tutor loaded with your notes, problem sets at each student's level, an interactive page instead of a static slide.

New ways to assess. Students who explain their own code to a quiz, a replication package the opponent runs, a defence that tests what the text cannot show.

Three jobs for an agent: feedback machine, talented TA, personal assistant

1 · Supervision

- A feedback skill students run on drafts
- Seminar instructions and checklists
- Model defence questions for opponents
- A code check students run first

Mental model: a feedback machine.

2 · Course materials

- Slides with current examples
- Problem sets, solutions, rubrics
- Quiz banks from your notes
- Data exercises in Stata, R, Python

Mental model: a talented TA.

3 · Practical and admin

- Course memo in a new template
- Summaries of course evaluations
- Schedules and announcements
- A website for the course

Mental model: a personal assistant.

Admin is an important use case

Friction is highest. The cost of starting dominates the cost of doing.

The quality bar is different. No referee: a course memo has to be correct, not elegant.

There is nobody else to delegate to. Small tasks are not worth giving to someone else.

Example: next year's course memo

A new template, new dates, new readings.

Without an agent. Copy, paste, reformat, update every date by hand.

With an agent. Drop last year's memo, the new template and this year's schedule in the folder. The agent fills it in, you check.

A task that is easy to verify and that nobody pays you for.

What does the drop in friction mean?

1. **Friction drops** in writing, editing and admin: the course page, the course memo.
2. **The bottleneck shifts** from producing to judging.
3. **Verification becomes the differentiator.** Not typing speed, not technical fluency.
4. **Tooling will keep moving.** The principles, specificity, iteration, verification, will not.

4 · Skills

A skill is a markdown file the agent loads when you type its name

```
---
name: thesis-feedback
description: Assess a thesis draft against
  the criteria in rubric.md
---
Read the thesis in this folder and rubric.md.
For each criterion, quote the evidence with
section references and name the one change
that would most improve the thesis.
Report, and do not rewrite the student's text.
Save the report as feedback.md.
```

It lives at `.claude/skills/thesis-feedback/SKILL.md` in the project folder, or under `~/.claude/skills/` for every project. Typing `/thesis-feedback` makes the agent follow the file. The description also lets the agent pick the skill by itself when a request matches.

Skills can be much longer. `/review-paper`, from the skill library:

663

lines of markdown: about ten pages of instructions you can read, fork and edit

8

reviewers it launches, each with one job

Three ways to create a skill: write it, ask for it, or save a session that worked

Write it

A text file with a name, a one-line description and the instructions, in any editor.

When you already know the method.

Ask for it

“Turn rubric.md into a skill called thesis-feedback that assesses one criterion at a time, quotes the evidence and does not rewrite the text.”

The agent writes the file. You read it before you use it.

Save it

At the end of a session that went well: “Save what we just did as a skill, so I can run it again next week.”

The method you refined in conversation becomes reusable.

Anthropic’s own skill library includes `skill-creator`, a skill that writes skills and tests them on examples.

Skills follow an open format, Agent Skills, that Claude Code, Codex and GitHub Copilot all read.

A skill gets better by testing: run it, read the output, fix the instruction, run it again

1. **Run it** on a case you know well, such as the thesis example from section 2.
2. **Read the output** against what you know, and mark the comment you disagree with.
3. **Fix the instruction** that caused it. Edit the line in the skill, not the report.
4. **Run it again** in a fresh conversation, and keep each version so a change can be undone.

One instruction, before and after

Before. “For each criterion, list the weaknesses.”

After. “For each criterion, quote the evidence first. Then name at most one weakness, with a section reference. If the criterion is met, say so.”

An illustrative edit. A report that only lists weaknesses reads harsher than the thesis deserves.

Or ask: “Update the thesis-feedback skill so that it says when a criterion is met.” The agent edits the file, and you read what it changed.

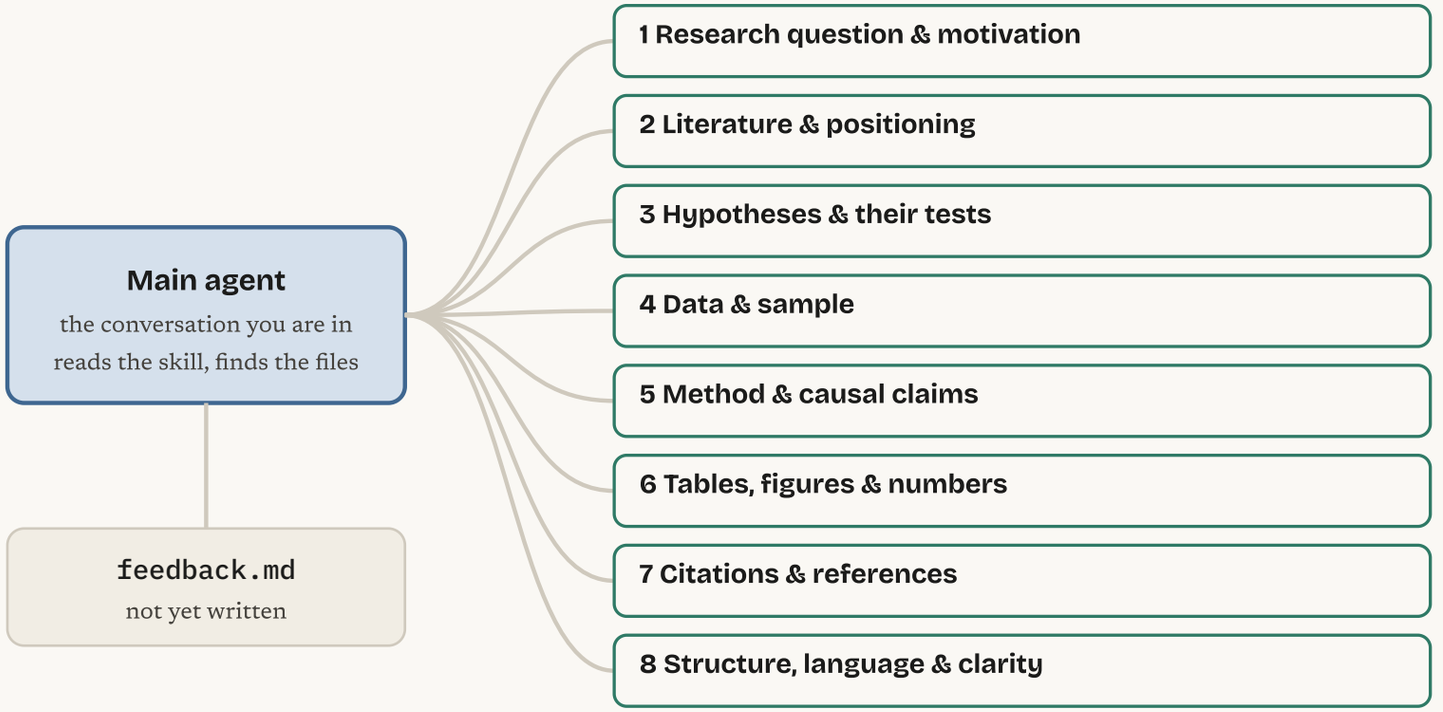
The skill you assess with can be the skill students check with

Give it to students

Before each supervision meeting, students run `/thesis-feedback` on their own draft and bring the report. The criteria stop being a mystery.

Teach them to write their own. A skill says what to check, in what order, and what counts as evidence. One assignment: write a skill that checks one rubric criterion, run it on your own draft, and hand in the skill with a paragraph on what it missed.

Eight narrow reviewers beat one diffuse reader



Run eight reviewers | Reset | Press **run**: the main agent sends the thesis to all eight at once.

Stylised: a rubric split into eight criteria, one reviewer each, the way /review-paper splits a paper; the timing is stylised. Teal: a reviewer with one job. The strip along each row is its window: a grey stub, the whole thesis in brick, the rest empty. Blue: the main agent, which finds the files and consolidates.

5 · Replication packages and coding

Agents are really good at coding, even without agentic capabilities

Galiani et al. (2026) benchmark applied econometric tasks in Stata, R and Python. Moving from a chatbot that writes one script to an agent that runs and revises its own code is what raises task success.

Under the chatbot, results differ a lot by software. Under the agent, the differences largely disappear.

74 → 96%

task success, chatbot to constrained agent

8 cents

extra cost per run

Galiani et al. (2026), NBER working paper 35588, abstract. “Constrained agent”: one that runs its code and fixes errors, but within a fixed set of tools.

A student's replication package is a README, a master script and portable paths

1. A README: data sources, access terms, software versions, runtime.
2. A table mapping each table and figure to the script that makes it and the file it writes.
3. One master script that runs the pipeline in order, with relative paths and seeds set.
4. The data the student may share, and access instructions for the data they may not.

Three prompts, then one check

"Read `Code/` and draft the README, including the exhibit-to-script table."

"List every input file the code reads; flag the ones not in this folder."

"Make every path relative and write `master.do`."

Then verify: *the opponent runs it in a fresh folder and compares the tables with the thesis. If the data cannot be shared, the code and the README are the package.*

The agent's thesis is close to a package already: one script runs every step

```
Code/
  run_all.sh              runs every step in order
  00_programs.do          shared Stata helpers
  01_download_dst.py      14 tables, Statistics Denmark
  02_download_bm010.py    prices, Finans Danmark
  03_download_external.py  ILO scores, boundaries
  04_build_geo.py
  10_build_exposure.do
  11_build_panel.do
  20_descriptives.do
  21_eventstudy.do
  logs/                   one log per Stata step
Data/raw/  Data/clean/    not shared
Figures/  Tables/  AI_tools/research_plan.md
```

Already there. One script that rebuilds everything from the download, numbered steps, relative paths, a log per step, and raw files saved with their download date.

Missing. A README with the data sources and access terms, software versions and run time, and the table that maps each figure and table to the script that makes it.

The test. A fresh agent in an empty folder, given only the README, tries to run it.

What to tell the students about verification?

The author is responsible for the output, and has to make sure that it is right

Agents are fast, cheap and competent at code, including in languages you do not know.

> **The errors that matter are not code crashes**
>
> Agents are very good at finding and fixing what crashes. Had an agent used the wrong sample in a table, or the wrong answer in a solution key, the output would still look plausible, but it would be wrong.

claesbackman.com/verifying-llm-output; Litt (2026); Goldsmith-Pinkham (2026); Scott Cunningham.

First principle: read whatever goes into your paper or in front of your students

The code is the analysis. Once a result is in the paper, you are responsible for it.

Not all code is equal. You don't need to know the website code, but you should probably check the regressions.

The answer key is the exam. A wrong key marks right answers wrong, and the student who trusts it learns the wrong thing.

A diff is a smaller thing to read than a codebase

A reviewer that sees everything

Spreads its attention thin, gives generic answers, and spends the window on files that did not change.

A reviewer that sees the diff

Knows exactly what is new and compares it against code you already checked.

You can use `git diff`, or your agent can. A *diff* is the list of lines that changed since the last saved version.

Version control is a verification tool. This is a good reason to use Git.

Different levels of checks

CHECK	WHAT YOU DO	WHAT IT CATCHES	COST
1 · Fresh agent reviews the diff	A fresh session or subagent reviews the diff and reports	Errors invisible to the agent that wrote the code	Minutes
2 · Different model	Same diff, model from another developer (Codex vs. Claude)	Shared habits: two copies of one model make the same mistakes	Minutes
3 · Make the agent quiz you	Fresh session explains the change, then quizzes you	Your own understanding gap	10–15 min
4 · Reimplement in another language	Fresh folder, `spec.md` and `README.md` only, other language and model, you run it	Code that faithfully implements something other than the paper	Hours

Cheap ones first, the expensive one for the main tables. Checks 2 to 4 each have a room in the appendix; click the name.

6 · Risks: hallucination, copyright and plagiarism

Three worries, each with a habit that answers it

Hallucination

Invented citations, numbers and quotes are the token stage from section 1. You need to check.

Habit: check every reference and number against its source.

Copyright

What you upload goes to the provider, and can be used for model training (e.g. under a free plan). Course literature, a colleague's slides and a student's thesis are not yours to upload by default.

Habit: upload your own material, or material you have permission to use, in a tool KTH has an agreement for.

Plagiarism

A model can reproduce text it was trained on, and a student can hand in text they did not write.

Habit: say what is allowed, and ask the student to explain the passage.

Six things to try this week

Update a lecture

Slides with 2015 examples get current ones; the structure stays.

Write a rubric

From the course's learning outcomes. Then test it on the thesis example.

Build a data exercise

From a public API: ECB, SCB, Eurostat, World Bank. Plan first.

A quiz bank

From one week's notes. Check every answer key yourself.

Feedback on a draft

The thesis example, against your rubric. Then give the skill to students.

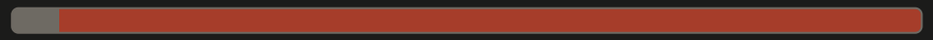
Mock exams and problem sets

Your students may already be doing this.

The end (or the beginning?)

Later we will try out practical workflows together.

Slides, skills, guides: claesbackman.com/ai_workshop



Claes Bäckman, SAFE

Appendix

FROM	SLIDE
1 · Fundamentals	Providers trade places
1 · Fundamentals	The default model is enough
1 · Fundamentals	Effort spends the token budget
1 · Fundamentals	The surface is your choice
1 · Fundamentals	Claude Code and Codex
1 · Fundamentals	The six tools the harness adds
1 · Fundamentals	Four good habits
1 · Fundamentals	Signs of session drift

FROM	SLIDE
1 · Fundamentals	Which file formats need converting
1 · Fundamentals	Converting is one sentence
3 · Teaching	A data exercise from the ECB
3 · Teaching	What the ECB prompt is testing for
4 · Skills	CLAUDE.md starter template
4 · Skills	My skill library
5 · Replication	Check 4: reimplement elsewhere
5 · Replication	Four checks, the full table
	References

Providers trade places

The frontier labs

Claude, ChatGPT and Gemini have each led at some point in the last year. Grok (xAI) and Llama (Meta) compete. The leading provider changes with each release.

Open weights

Cheaper to run, and many are not far behind on performance. Mistral is European, which can matter when a data-protection office asks where the model runs.

Switching costs are low. It is more important to start.

The default model is enough for daily work

Fast, cheap	`Haiku`	Conversions, summaries, bulk tasks
Default	`Sonnet`	Daily coding, editing, drafting
Frontier	`Opus`	Hard reasoning, adversarial review
Most capable, slowest	`Fable`	Long-running agents, the hardest tasks; costs and waits more

Claude's model tiers as of August 2026.

Effort spends the same token budget as files

Effort sets how many tokens the model spends thinking before it answers. It is a setting in the tool, next to the model choice.

Turn it up for deep analysis and adversarial review. Turn it down for speed on routine tasks.

3–4
characters of English per token

15–30k
tokens in a short paper

Tokens are the unit models bill and reason in. Thinking tokens and file tokens come out of the same window.

Rules of thumb.

The surface is your choice; the workflow is the same

SURFACE	WHAT IT IS GOOD FOR
VS Code	What I use. File tree, diff view, integrated terminal, side-by-side PDF preview.
Terminal	`claude` or `codex` in any shell. Good for servers and quick one-shots.
Desktop app	Claude Desktop's *Cowork* and *Code* tabs, or the Codex desktop app.
Web, cloud	claude.ai/code or chatgpt.com/codex. Point at a GitHub repository.
Mobile	Capture ideas; monitor or redirect cloud agents. Not for editing.
Other editors	Cursor, JetBrains, Zed. Keep your editor, add the agent.

More at claesbackman.com/agentic-ai-overview.

Claude Code and Codex: different names for the same workflow

All three read your files, run code, edit drafts and iterate. A *skill* is a saved prompt, invoked by name.

Claude Code (Anthropic)

Standing instructions: `CLAUDE.md`
Skills: run with `/review-paper`
Backed by: Fable, Opus, Sonnet, Haiku
Surfaces: VS Code, CLI, web, desktop

Codex (OpenAI)

Standing instructions: `AGENTS.md`
Skills: run with `$review-paper`
Backed by: GPT family
Surfaces: VS Code, CLI, web, mobile

The harness adds six things the model cannot do alone

CAPABILITY	WHAT IT LETS THE AGENT DO
Project files (`Read`)	Read files, inspect folders, understand the project structure
Editing (`Edit`, `Write`)	Propose and apply changes; create or modify files
Shell (`Bash`)	Run commands such as `Rscript`, `pdflatex`, tests, linters
Search (`Grep`, `Glob`)	Search file names and file contents in the project
Web (`WebSearch`)	Search or fetch web pages when web access is enabled
MCP and plugins	Connect to external tools (GitHub, databases, custom servers) through a standard interface, the model context protocol

The names are Claude Code's; Codex has the same six under other names. Back to the loop or the appendix overview.

Four good habits

Point at files	`@filename`	Name the file you mean; do not let the agent read everything in the folder.
One task, one session	`/clear`	Start a new session for a new task. Thirty turns or a shifted task: restart. Signs of drift.
Watch the meter	`/context`	Shows what is filling the window and by how much.
Convert once	`/pdf-to-markdown`	Long PDFs become markdown once, up front; the markdown is committed and pointed at from then on. Which formats need it.

The first three are Claude Code commands, with equivalents in Codex; /pdf-to-markdown is a skill from the library. The starter CLAUDE.md template is in the appendix.

Signs of session drift

1. Claude ignores `CLAUDE.md`
2. Repeats mistakes you already corrected
3. Invents files that do not exist
4. Gives circular answers
5. Forgets the last three turns

Fix. Start a fresh session or close and reopen. Re-state the task. Re-attach the relevant file.

Thirty turns, or the task has shifted: start fresh. Always.

Back to the habits or the appendix overview.

Text formats are free; binaries cost a conversion

Native, no friction	`.md`, `.tex`, `.txt`, `.bib`, `.csv`, `.json`, `.yaml`, and code: `.py`, `.R`, `.do`, `.ipynb`
Convert once	`.docx`, born-digital `.pdf`, `.xlsx`: to markdown or CSV, saved next to the original
Read through code, not as text	Data binaries such as `.dta` and `.rds`: the agent loads them in Stata, R or pandas and looks at the output
Hard: OCR or skip	Scanned PDFs, survey exports such as `.qsf`

Convert once, commit the markdown alongside the original, point the agent at the markdown.

Back to the habits, on to converting, or the appendix overview.

Converting is one sentence to the agent

“Convert every PDF in `papers/` to markdown, save each next to the original, and tell me which ones came out garbled.”

Back to the file formats or the appendix overview.

For prompting: make a plan, document the work, and get something you can check

> *Using the ECB Data Portal API, download two quarterly series from 2000 onwards: the euro-area residential property price index and the deposit facility rate averaged to quarterly. Look up both series keys first and show me the keys and your planned requests before you download anything.*
>
> *Save the merged series as* `data/raw/ecb_hp_rate.csv` *and write* `data/README.md` *giving each series key, its units, and its transformation. Use Python.*
>
> *Then plot the two series together for a week 3 exercise, with three questions for students and a separate solutions file. Report the 2008Q1 and 2022Q1 values of each so I can check them against the portal.*

Three paragraphs, three moves: plan first, document, verify. What the prompt is testing for is in the appendix.

What the ECB prompt is testing for

1. **What an API is.** A URL that returns data as CSV or JSON instead of a web page. The ECB Data Portal serves every published series with no registration and no key.
2. **The series key is the whole trick.** A series is addressed by a dotted key such as `RPP.Q.I9.N.TD.00.3.00`. The agent must look the key up before it can fetch anything, which is why the prompt asks to see the keys first.
3. **Same pattern elsewhere.** FRED, Eurostat, SNB, BIS, World Bank, OECD.
4. **Verify.** Agents invent series keys that look plausible and return nothing, or the wrong series. Check a few values against the portal's own interface.

Back to the prompt or the appendix overview.

CLAUDE.md starter template

```
# About me
I teach [course] and supervise theses in [field].

# How I want you to work with me
- Ask clarifying questions before generating long output.
- Critical, skeptical tone in feedback. Do not flatter.
- When editing, preserve my voice (see voice.md).
  No generic LLM phrasing.
- Avoid bullet lists and passive voice in formal writing.
- Cite the specific line or section you are commenting on.

# Things to avoid
- Do not fabricate citations.
- Do not over-claim causality.
- Do not insert emoji or markdown decorations in formal
        docs.
```

Always loaded, every session. Standing instructions, the things you would otherwise repeat in every prompt.

Session 2 builds one for your own project with `/init`.

Back to the harness, the habits or the appendix overview.

My skill library

SKILL	USE
`/review-paper`	Pre-submission referee report, 8 agents
`/review-paper-light`	Fast 2-agent version, about 1 minute
`/review-paper-code`	Paper and code alignment review
`/review-grant`	Pre-submission grant review, 6 agents
`/review-pap`	Pre-analysis-plan review
`/audit-analysis`	Adversarial audit of changed analysis code
`/explain-diff`	Explain a code change as an HTML page with a quiz
`/paper-version`	Paper to policy brief, 1-page or 5-page summary
`/pdf-to-markdown`	Convert PDFs to readable markdown

All at github.com/claesbackman/AI-research-feedback (MIT), linked from claesbackman.com/ai_workshop. Fork them, modify them, make them yours. Back to check 3 or the appendix overview.

Check 4: reimplement the analysis in another language

What and why

If your code is Stata, a second agent writes the results in Python or R from the paper's description alone. An idea from Scott Cunningham.

If coding slips are independent across languages, differences in the output reveal the errors.

How to isolate it

New folder, fresh session, a `spec.md` copied from the paper with the results deleted, a `README.md` describing the raw data.

This is a major enterprise, so reserve it for the main tables and for things that matter.

Back to check 3 or the appendix overview.

Four checks, the full table

CHECK	WHAT YOU DO	WHAT IT CATCHES	COST
Fresh agent attacks the diff	A fresh chat or subagent reviews the diff	Errors invisible to the agent that wrote the code	Minutes
Different model	Same diff, model from another developer (Codex vs. Claude)	Shared habits: two copies of one model make the same mistakes	Minutes
Make the agent quiz you	Fresh session explains the change, then quizzes you	Your own understanding gap	10–15 min
Reimplement in another language	Fresh folder, `spec.md` and `README.md` only, other language and model, you run it	Code that faithfully implements something other than the paper	Hours

Back to the four checks or the appendix overview.

References

Bryan, Kevin A. 2026. *Eight Rules for Teaching in AI World*. Https://kevinbryanecon.com/teachwithai.html.

Galiani, Sebastian, Federico Ariel López, and Raul A Sosa. 2026. *AI Agents and Prompt Engineering in Econometric Coding*. National Bureau of Economic Research.

Higher Education Policy Institute and Kortext. 2026. *Student Generative Artificial Intelligence Survey 2026*. HEPI Report No. 199. Higher Education Policy Institute. https://www.hepi.ac.uk/reports/student-generative-ai-survey-2026/.

Jabarian, Brian, and Alex Imas. 2025. *Artificial Writing and Automated Detection*. NBER Working Paper No. 34223. National Bureau of Economic Research. https://doi.org/10.3386/w34223.

Malmström, Hans, Christian Stöhr, and Amy Wanyu Ou. 2023. *Chatbots and Other AI for Learning: A Survey of Use and Views Among University Students in Sweden*. Chalmers Studies in Communication and Learning in Higher Education 2023:1. Chalmers University of Technology. https://research.chalmers.se/en/publication/535715.

Mollick, Ethan. 2026. *A Guide to Which AI to Use in the Agentic Era*.

Novy-Marx, Robert, and Mihail Velikov. 2026. "Artificial Intelligence–Powered (Finance) Scholarship." *Journal of Economic Literature* 64 (1): 5–37.

OpenAI. 2023. *New AI Classifier for Indicating AI-Written Text*. https://openai.com/blog/new-ai-classifier-for-indicating-ai-written-text/.

Velikov, Mihail. 2026. *EDHEC Master Class – Using AI for Research*.